UNIVERSIDAD NACIONAL MICAELA BASTIDAS DE APURÍMAC

Facultad de Ingeniería

Escuela Académico Profesional de Ingeniería Informática y Sistemas

Informe del curso de Inteligencia Artificial II

Clasificación de implementos de seguridad en obras mediante DenseNet121

Presentado por: Jimmy Vargas Huarcaya

Docente: Ing. Manuel Ibarra Cabrera

Abancay, Perú

2026

ÍNDICE

1. Introducción 2

2. Objetivo 2

2.1. General 2

2.2. Específicos 2

3. Fundamentos teóricos 3

3.1. Redes neuronales convolucionales 3

3.2. Arquitectura DenseNet 3

3.3. DenseNet121 3

3.4. Aprendizaje por transferencia 4

3.5. Conceptos para interpretar el entrenamiento 4

4. Comparación con otras arquitecturas 4

5. Caso práctico 5

5.1. Dataset 5

5.2. Herramientas utilizadas 6

5.3. Preparación de los datos 6

5.4. Implementación del modelo 7

5.5. Entrenamiento 8

5.6. Resultados 9

5.7. Análisis de resultados 10

6. Ventajas y limitaciones 11

6.1. Ventajas 11

6.2. Limitaciones 12

7. Conclusiones y recomendaciones 12

7.1. Conclusiones 12

7.2. Recomendaciones 12

8. Bibliografía 13

1. Introducción

La clasificación de imágenes permite asignar una categoría a lo que aparece en una fotografía. En este trabajo se aplica esta tarea a regiones de imágenes relacionadas con la seguridad en obras: cabeza, casco, chaleco y trabajador. El propósito es estudiar la arquitectura DenseNet mediante un caso práctico que se pueda entrenar, evaluar y demostrar en una computadora personal.

Se utilizó DenseNet121 con pesos previamente entrenados en ImageNet. En lugar de comenzar desde cero, se conservaron sus capas de extracción de características y se adaptó el clasificador final a las cuatro categorías del proyecto. Esta decisión permitió realizar el entrenamiento con una tarjeta gráfica GTX 1650 Ti.

Los datos proceden de un conjunto público de Roboflow. Sus anotaciones se aprovecharon para recortar los objetos antes de clasificarlos. El modelo obtuvo una exactitud de 97,69 % en 1.081 recortes de prueba. El informe presenta la preparación de los datos, las decisiones del entrenamiento y los errores observados. Su alcance es la clasificación de recortes, no la localización automática de objetos dentro de una escena completa.

2. Objetivo

2.1. General

Implementar y evaluar un modelo basado en DenseNet121 para clasificar recortes de cabeza, casco, chaleco y trabajador en imágenes relacionadas con la seguridad en obras.

2.2. Específicos

• Preparar un conjunto de recortes etiquetados y separar los datos de entrenamiento, validación y prueba.

• Adaptar el clasificador de DenseNet121 a las cuatro categorías mediante aprendizaje por transferencia.

• Entrenar el clasificador con GPU y seleccionar el modelo según su desempeño en validación.

• Evaluar los aciertos y errores del modelo y presentar una demostración con el archivo entrenado.

3. Fundamentos teóricos

3.1. Redes neuronales convolucionales

Una red neuronal convolucional, o CNN, procesa una imagen por etapas. Las primeras capas suelen identificar patrones sencillos, como bordes y cambios de color. Las capas posteriores combinan esa información para formar representaciones más útiles. Finalmente, un clasificador utiliza esas características para decidir a qué categoría pertenece la entrada.

En una tarea de clasificación se entrega una imagen y se obtiene una categoría. En una tarea de detección también se necesita indicar dónde se encuentran los objetos. Por eso, reconocer un casco ya recortado no es la misma tarea que encontrar todos los cascos en una fotografía de una obra.

3.2. Arquitectura DenseNet

DenseNet fue presentada por Huang et al. (2017). Su idea principal es que, dentro de un bloque denso, cada capa pueda utilizar las características producidas por las capas anteriores. Así, la información no tiene que pasar únicamente de una capa a la siguiente: puede reutilizarse en varias etapas.

Estas conexiones combinan características mediante concatenación, es decir, las reúnen conservando sus componentes. Los bloques de transición reducen el tamaño de la información entre bloques densos. Esta organización favorece la reutilización de características y ayuda a transmitir la señal durante el aprendizaje (Huang et al., 2017).

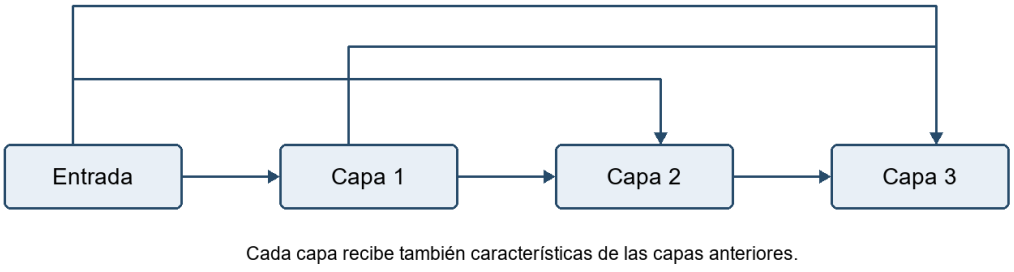


Figura 1. Conexiones simplificadas dentro de un bloque denso (Huang et al., 2017).

3.3. DenseNet121

DenseNet121 es una variante de esta arquitectura. Su implementación contiene cuatro bloques densos con 6, 12, 24 y 16 unidades, respectivamente. Después de extraer las características, resume la información y la entrega al clasificador. En este proyecto, el clasificador recibe 1.024 características y produce cuatro salidas, una por categoría (PyTorch, s. f.-a).

3.4. Aprendizaje por transferencia

El aprendizaje por transferencia aprovecha una red que ya fue entrenada con otras imágenes. Se parte de sus pesos aprendidos y se adapta la parte necesaria a la nueva tarea. Una opción consiste en mantener fija la base de la red y entrenar solamente el clasificador, como se explica en el tutorial de PyTorch (s. f.-b).

Se eligió esa opción para este trabajo. La base de DenseNet121 quedó congelada: sus pesos no cambiaron durante el entrenamiento. Solo se ajustaron los 4.100 parámetros del clasificador final. Por tanto, el experimento corresponde a una adaptación de una red preentrenada y no al entrenamiento completo de DenseNet desde cero.

3.5. Conceptos para interpretar el entrenamiento

Una época es un recorrido completo por los datos de entrenamiento. El tamaño de lote indica cuántos ejemplos se procesan juntos; en este caso se utilizaron ocho. La tasa de aprendizaje controla el tamaño de los ajustes que realiza el optimizador. Se usó Adam con una tasa de 0,0005.

La pérdida resume qué tan equivocadas están las salidas del modelo respecto a las etiquetas. Se utilizó entropía cruzada porque cada recorte pertenece a una de cuatro clases. Una pérdida menor indica un mejor ajuste según esta función, pero no garantiza por sí sola un mejor resultado en fotografías nuevas.

La exactitud expresa la proporción total de aciertos. La precisión de una clase indica cuántas de sus predicciones fueron correctas; el recall indica cuántos ejemplos reales de esa clase fueron reconocidos. F1 combina precisión y recall. El F1 macro promedia el F1 de las cuatro clases con el mismo peso, aunque tengan distintas cantidades de ejemplos.

4. Comparación con otras arquitecturas

La comparación de la tabla 1 permite ubicar DenseNet frente a otras CNN. Es una comparación de sus ideas principales; VGG y ResNet no fueron entrenadas en este proyecto.

Tabla 1. Comparación conceptual de arquitecturas CNN (Simonyan y Zisserman, 2015; He et al., 2016; Huang et al., 2017).

Arquitectura	Idea principal	Diferencia relevante
VGG	Capas convolucionales organizadas de forma secuencial.	Estructura sencilla de seguir; sus versiones clásicas utilizan clasificadores grandes.
ResNet	Conexiones que suman la entrada a una transformación aprendida.	Facilita el entrenamiento de redes profundas mediante conexiones residuales.
DenseNet	Reúne características de capas anteriores dentro de cada bloque.	Favorece la reutilización de información mediante concatenación.

5. Caso práctico

5.1. Dataset

Se utilizó PPE Compliance Detection, versión 5, publicado por Yihong Chen en Roboflow Universe bajo licencia CC BY 4.0. El archivo descargado contiene 2.000 imágenes y anotaciones de cuatro clases: Head, Helmet, Vest y Worker. Las anotaciones señalan la ubicación de cada objeto mediante una caja. La procedencia y la licencia se conservaron en los archivos del proyecto (Chen, s. f.).

Una fotografía puede incluir varios objetos. Por ejemplo, de una misma imagen se pueden obtener un recorte de casco, otro de chaleco y otro de trabajador. Por ese motivo, el número de recortes es mayor que el de fotografías originales. Las categorías describen el contenido de la región anotada; un recorte de trabajador puede incluir también su casco o chaleco.

Tabla 2. Distribución de los recortes utilizados (Vargas Huarcaya, 2026).

Clase	Entrenamiento	Validación	Prueba	Total
Cabeza	1023	223	90	1336
Casco	1130	486	307	1923
Chaleco	1290	398	287	1975
Trabajador	2153	709	397	3259
Total	5596	1816	1081	8493

La división original fue de 1.400 imágenes de entrenamiento, 400 de validación y 200 de prueba. Se mantuvo esa división al preparar los recortes, excluyendo imágenes duplicadas entre conjuntos. El conjunto final contiene 8.493 recortes: 5.596 para entrenar, 1.816 para validar y 1.081 para la prueba.

Entrenamiento sirve para ajustar los pesos. Validación permite elegir el modelo y decidir cuándo detener el proceso. Prueba se usa para medir el desempeño del modelo seleccionado. Estos conjuntos cumplen funciones diferentes y no deben mezclarse.

5.2. Herramientas utilizadas

El trabajo se desarrolló en Windows con Visual Studio Code. El equipo cuenta con un procesador Ryzen 7 4800H y una GPU NVIDIA GTX 1650 Ti de 4 GB. Se creó un entorno virtual para mantener las dependencias separadas de otros proyectos.

Tabla 3. Herramientas del proyecto (Vargas Huarcaya, 2026).

Herramienta	**Uso**
Python 3.12	Lenguaje de programación.
PyTorch 2.7.1 y torchvision 0.22.1	Carga de DenseNet121, entrenamiento y predicción.
CUDA 11.8	Uso de la GPU durante el entrenamiento.
Pillow y NumPy	Lectura y preparación de imágenes.
Matplotlib y scikit-learn	Gráficas y métricas de evaluación.
Git y GitHub	Registro de cambios y publicación del proyecto.

5.3. Preparación de los datos

Primero se conservó una copia de las imágenes y etiquetas originales. Luego se revisaron las imágenes repetidas entre conjuntos. Se excluyeron 30 imágenes duplicadas o casi idénticas y un recorte menor de cuatro píxeles. Esto ayuda a evitar que el modelo sea evaluado con copias de imágenes utilizadas para entrenar.

Las cajas anotadas se convirtieron en recortes RGB y se organizaron por clase. Durante el entrenamiento, cada recorte se redimensionó a 128 × 128 píxeles y se normalizó con los valores de ImageNet. Se eligió este tamaño para reducir el trabajo de procesamiento en la GPU disponible. No se aplicaron aumentos de datos en esta ejecución.

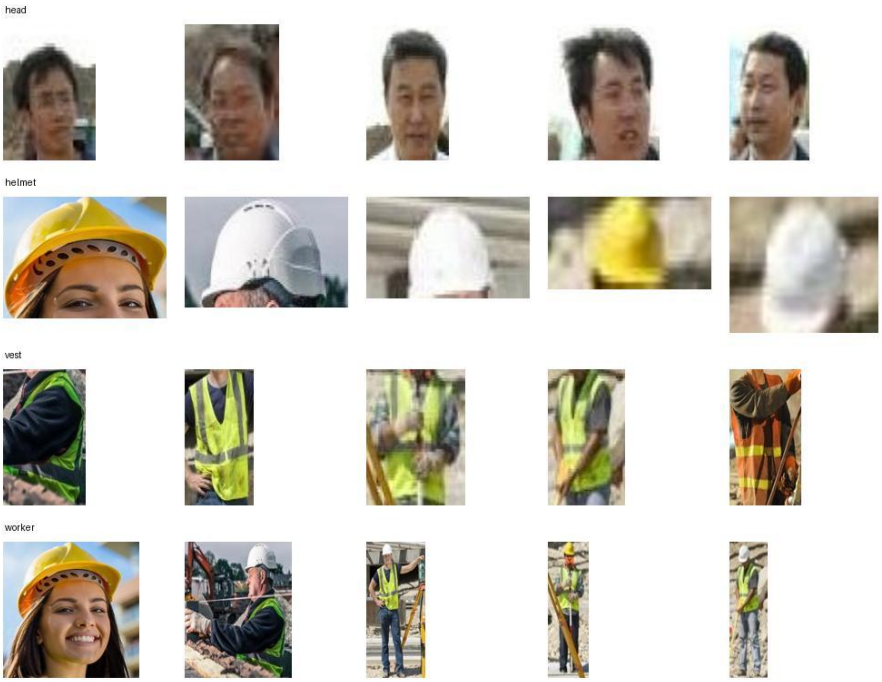


Figura 2. Ejemplos de las cuatro categorías del dataset (Chen, s. f.; CC BY 4.0).

5.4. Implementación del modelo

Se cargó DenseNet121 con los pesos ImageNet disponibles en torchvision. Después se congelaron los parámetros de la base y se reemplazó su clasificador por una capa de cuatro salidas. La base, incluidas sus estadísticas de normalización, permaneció en modo de evaluación durante el ajuste del clasificador.

El recorrido de una imagen es el siguiente: entra un recorte RGB, la base DenseNet extrae sus características y el clasificador asigna una puntuación a cada categoría. Para predecir, se elige la categoría con la puntuación más alta. La función softmax permite mostrar esas puntuaciones en una escala entre cero y uno; no representan una garantía de acierto.

Tabla 4. Componentes del modelo adaptado (Vargas Huarcaya, 2026).

Componente	**Configuración**
Entrada	Recorte RGB de 128 × 128 píxeles.
Base	DenseNet121 preentrenada en ImageNet, congelada.
Características de entrada al clasificador	1.024 valores por recorte.
Clasificador	Capa lineal con cuatro salidas.
Parámetros totales	6.957.956.
Parámetros ajustados	4.100, correspondientes al clasificador.

El código se organizó en archivos separados para preparar los datos, comprobar la GPU, entrenar y realizar predicciones. Los cambios se registraron con Git y se publicaron en el repositorio del curso:

https://github.com/JIMdvd/densenet-ppe-ia2

El modelo entrenado se encuentra en la sección Releases. Para la demostración se utiliza el archivo densenet121_ppe.pt, sin repetir el entrenamiento. El script demo.py ejecuta cuatro ejemplos del conjunto de prueba y guarda sus predicciones como imágenes y archivos JSON.

5.5. Entrenamiento

Antes de la ejecución definitiva se comprobó el funcionamiento de CUDA y se realizó un ensayo de cinco épocas. Una revisión posterior identificó más duplicados visuales entre conjuntos. Se corrigieron los datos y se reinició el entrenamiento desde los pesos ImageNet, con un clasificador nuevo. Los resultados de ese ensayo preliminar no se mezclan con los resultados finales.

Tabla 5. Configuración del entrenamiento definitivo (Vargas Huarcaya, 2026).

Parámetro	**Valor**
Optimizador	Adam
Tasa de aprendizaje	0,0005
Función de pérdida	Entropía cruzada
Tamaño de lote	8 recortes
Máximo de épocas	20
Parada temprana	4 épocas sin reducir la pérdida de validación al menos 0,001
Semilla	42
Épocas ejecutadas	9
Modelo seleccionado	Época 5, por menor pérdida de validación

La parada temprana permite finalizar el proceso cuando la validación deja de mejorar. Se conserva el mejor modelo, no necesariamente el último. La ejecución acumuló aproximadamente 6 minutos y 23 segundos de entrenamiento y validación; ese tiempo no incluye la carga inicial, el guardado de archivos ni la evaluación final.

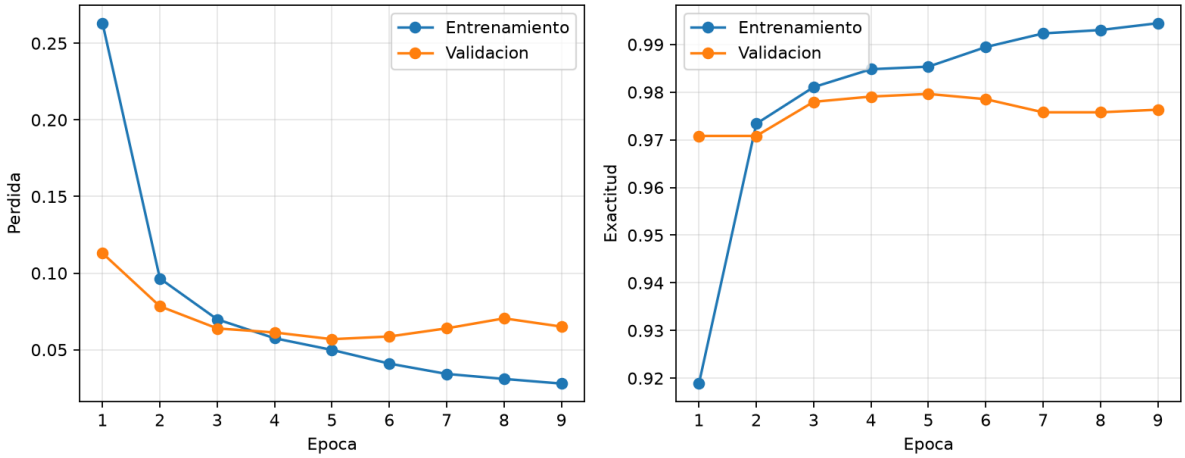


Figura 3. Evolución de la pérdida y la exactitud durante nueve épocas (Vargas Huarcaya, 2026).

5.6. Resultados

El modelo seleccionado se evaluó con 1.081 recortes de prueba. Clasificó correctamente 1.056 y se equivocó en 25. La exactitud fue de 97,69 % y el F1 macro de 97,25 %. Como referencia sencilla, predecir siempre la clase trabajador habría obtenido 36,73 % de exactitud en este conjunto.

Tabla 6. Resultados por categoría en el conjunto de prueba (Vargas Huarcaya, 2026).

Clase	Precisión	Recall	F1	Recortes
Cabeza	96,59 %	94,44 %	95,51 %	90
Casco	98,69 %	98,37 %	98,53 %	307
Chaleco	97,19 %	96,52 %	96,85 %	287
Trabajador	97,51 %	98,74 %	98,12 %	397

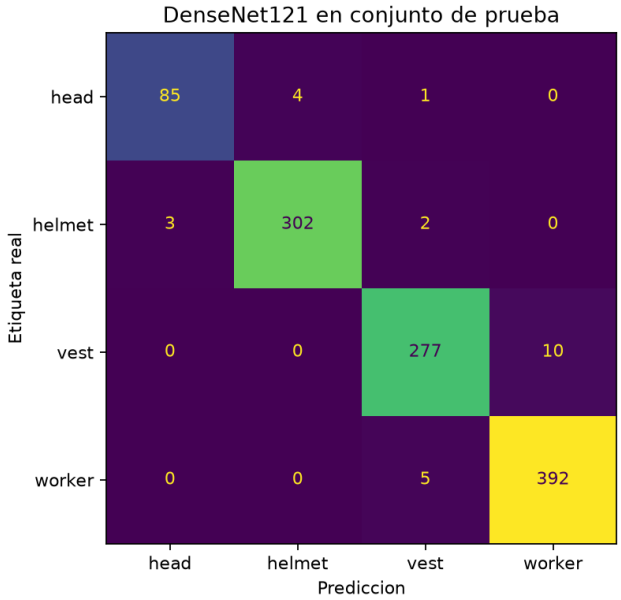


Figura 4. Matriz de confusión del modelo seleccionado (Vargas Huarcaya, 2026).

En la matriz, cada fila representa la etiqueta real y cada columna la predicción. Los valores de la diagonal son los aciertos. Los nombres head, helmet, vest y worker corresponden a cabeza, casco, chaleco y trabajador, respectivamente.

5.7. Análisis de resultados

Las curvas muestran una reducción rápida de la pérdida al inicio. La menor pérdida de validación se alcanzó en la época 5, con 0,0569. Después, el error de entrenamiento siguió bajando, pero el de validación aumentó. Esta diferencia sugiere que continuar ajustando el clasificador ya no aportaba una mejora general y podía favorecer el sobreajuste.

No se eligió el modelo por el número de épocas ni por su exactitud de entrenamiento. Se eligió por su pérdida de validación. Por eso se conservó la época 5 aunque el proceso llegó hasta la 9. La exactitud de validación del modelo elegido fue de 97,96 %.

La confusión más frecuente ocurrió entre chaleco y trabajador: diez chalecos se clasificaron como trabajador y cinco trabajadores como chaleco. Una explicación posible es que ambas

regiones comparten partes del cuerpo y prendas similares. Esta interpretación es coherente con los ejemplos, aunque no demuestra por sí sola la causa de cada error.



Figura 5. Ejemplos de aciertos y errores en las cuatro categorías (Vargas Huarcaya, 2026; imágenes de Chen, s. f., CC BY 4.0).

La clase cabeza obtuvo el recall más bajo, con 94,44 %, y contó con 90 ejemplos de prueba. Esto indica que el desempeño no es idéntico en todas las categorías. El resultado global es favorable para este conjunto, pero requiere nuevas pruebas antes de aplicarlo en otros escenarios.

6. Ventajas y limitaciones

6.1. Ventajas

El aprendizaje por transferencia permitió aprovechar características ya aprendidas por DenseNet121 y ajustar un clasificador pequeño. Esto hizo posible realizar el experimento con la GPU disponible. La separación del código en scripts también facilita volver a ejecutar la demostración y revisar cada etapa del trabajo.

Se conservaron la configuración, el historial y las predicciones de prueba. Estos archivos permiten comprobar los resultados, identificar errores y relacionar las gráficas con los datos reales. El repositorio público reúne el código, el registro de cambios y el modelo descargable.

6.2. Limitaciones

El modelo recibe objetos recortados. No genera cajas sobre una fotografía completa ni comprueba por sí solo si un trabajador utiliza correctamente todo su equipo de protección. Su salida está limitada a las cuatro clases del dataset.

La evaluación se hizo por recorte. Varios recortes pueden proceder de una misma fotografía y pueden existir escenas relacionadas que el filtro de duplicados no identifica. Además, las categorías tienen cantidades diferentes de ejemplos y las imágenes pequeñas pueden perder detalles al redimensionarse.

No se evaluaron fotografías nuevas de obras locales ni se compararon experimentalmente otras arquitecturas. Tampoco se ajustaron todas las capas de DenseNet ni se aplicaron aumentos de datos. Por ello, los resultados corresponden a la configuración y al conjunto utilizados, no a un rendimiento garantizado en cualquier imagen.

7. Conclusiones y recomendaciones

7.1. Conclusiones

Se implementó un clasificador de cuatro categorías con DenseNet121 mediante aprendizaje por transferencia. La preparación produjo 8.493 recortes y permitió mantener separados los datos utilizados para entrenar, validar y evaluar.

El modelo seleccionado obtuvo 97,69 % de exactitud y 97,25 % de F1 macro en prueba. Los 25 errores muestran que el reconocimiento no es perfecto. La mayor confusión se presentó entre chaleco y trabajador, mientras que cabeza tuvo el menor recall.

La parada temprana permitió conservar la época 5 y evitar elegir automáticamente el último modelo. El archivo entrenado puede utilizarse en la demostración sin volver a ajustar sus pesos. El código y sus resultados quedaron disponibles mediante Git y GitHub.

7.2. Recomendaciones

Se recomienda incorporar fotografías de otras obras y separar los conjuntos por escena o lugar de origen. También conviene ampliar los ejemplos de cabeza y revisar las confusiones entre trabajador y chaleco. En una siguiente etapa se podrían probar aumentos de datos y el ajuste de algunas capas de la base, utilizando validación para decidir si realmente mejoran el resultado.

Si se necesita localizar varios objetos en una imagen completa, será necesario añadir una etapa de detección. Esa ampliación debe evaluarse por separado y no confundirse con la clasificación de recortes desarrollada en este trabajo.

8. Bibliografía

Chen, Y. (s. f.). *PPE Compliance Detection* (versión 5) [Conjunto de datos]. Roboflow Universe. https://universe.roboflow.com/yihong-chen/ppe-compliance-detection-iln63/dataset/5

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. En *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 770-778). https://doi.org/10.1109/CVPR.2016.90

Huang, G., Liu, Z., van der Maaten, L., & Weinberger, K. Q. (2017). Densely connected convolutional networks. En *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 4700-4708). https://openaccess.thecvf.com/content_cvpr_2017/html/Huang_Densely_Connected_Convolutional_CVPR_2017_paper.html

PyTorch. (s. f.-a). *densenet121* [Documentación de torchvision]. https://docs.pytorch.org/vision/main/models/generated/torchvision.models.densenet121.html

PyTorch. (s. f.-b). *Transfer learning for computer vision tutorial.* https://docs.pytorch.org/tutorials/beginner/transfer_learning_tutorial.html

Simonyan, K., & Zisserman, A. (2015). *Very deep convolutional networks for large-scale image recognition.* International Conference on Learning Representations. https://arxiv.org/abs/1409.1556

Vargas Huarcaya, J. (2026). *DenseNet PPE IA2* [Código fuente y modelo entrenado]. GitHub. https://github.com/JIMdvd/densenet-ppe-ia2